

实践指南：如何与AI一起开发操作系统

理论讲了不少，现在该说说怎么实际操作了。

工作流程

第一步：理解任务

拿到任务别急着写提示词。先花时间把需求搞清楚：这个功能在系统中的位置是什么？和哪些模块交互？测试要检查什么？有哪些关键约束？

读实验指导书，读相关代码文件，看数据结构定义和辅助函数。测试代码也可以参考，那是最准确的需求说明。

第二步：梳理需求

理解了任务之后，试着用自己的话把需求说清楚：要实现或修改哪几个函数？输入输出是什么？正常情况怎么处理，异常情况怎么处理？有没有特殊的时序要求或者资源配对关系？对于修改任务，要明确修改的目标和预期效果是什么。

如果发现说不清楚某个地方，说明还没理解透，回到第一步继续深入。

第三步：编写提示词

按照四段式框架写提示词。[RELY]部分去代码里找真实定义，别自己编造。[GUARANTEE]部分列出所有函数签名，参数类型和返回值类型要和代码一致。[SPECIFICATION]部分把需求转化为Pre/Post-Condition、Success/Failure case的形式。

别为了简洁而省略关键信息。说清楚比说简洁更重要。

第四步：生成和验证

把提示词输入给AI，等它生成代码。观察一下是否按要求修改了文件，是否实现或修改了所有函数。

不要假设生成的代码就是对的。先读一遍，检查关键约束是否满足，错误处理是否完整。

然后编译。编译不过通常是[RELY]部分不完整或者和实际代码有出入。

编译过了就运行测试。测试失败别急着怪AI，先看是哪个测试失败了，测什么，为什么失败。然后回头看提示词，是不是某个需求没说清楚。

第五步：迭代优化

不一定能够一次成功。根据测试结果分析问题，如果问题较大，建议优化提示词；如果问题较小，可以直接和 AI 交互修改 Bug。

常见问题可能包括：需求遗漏（[SPECIFICATION]里补充）、需求不清（说得更具体）、边界情况遗漏（补充 Failure case）、系统约束遗漏（明确 Pre/Post-Condition）、实现逻辑错误（与 AI 分析原因并修改）。

每次修改提示词后重新生成，重新测试；如果是小问题，直接让 AI 修改后重新测试。反复几轮直到通过。

和AI协作

AI能根据提示词生成代码，但它不理解你的真实意图，不理解整个系统上下文，不会主动发现提示词的漏洞。所以别指望AI能"猜"，把需求说清楚说完整才是正道。

AI生成的代码不一定对。看到代码时问自己：这样实现符合系统约束吗？边界情况考虑到了吗？有没有资源泄漏？有没有竞态条件？你才是系统的最终负责人。

每完成一个模块，回顾一下哪些写法效果好，哪些地方容易出问题。积累经验，下次就能少踩坑。

提示词和代码要同步。手动改了代码或者发现提示词有问题，记得更新。提示词应该始终反映当前的需求。

开始写第一个提示词吧

AI驱动的开发本质上是不断"把需求说清楚"的过程。从模糊的功能描述到精确的行为规格，再到可运行的代码，再通过测试反馈优化，形成正向循环。

想要"说清楚"一个功能，远比想象中难。但正是这个难度，让你对操作系统的理解更深入。当你能用清晰、完整、无歧义的语言描述一个复杂系统功能时，你掌握的不只是"会写代码"，而是系统设计和需求分析能力。

现在，开始写第一个提示词吧。

Step by step, to the light of AI-driven OS development!